Time Optimization Algorithms for Traffic Systems Around Marshall University Using Bellman-Ford and Particle Swarm Optimization

Bethany Ledford
Department of Computer Sciences and Electrical Engineering, Marshall University
Ledford20@marshall.edu

Kendra Adkins
Department of Computer Sciences and Electrical Engineering, Marshall University
Adkins1783@marshall.edu

Emily Kisor
Department of Computer Sciences and Electrical Engineering, Marshall University
Kisor5@marshall.edu

Alec Bruce
Department of Computer Sciences and Electrical Engineering, Marshall University
Bruce39@marshall.edu

Abstract– This study presents a comparative analysis of traffic optimization algorithms applied to the street network surrounding Marshall University, focusing on the Bellman-Ford algorithm and Particle Swarm Optimization (PSO). Motivated by increasing congestion and pedestrian safety concerns, the research addresses the limitations of conventional traffic systems in dynamic university environments. While Bellman-Ford provides efficient static pathfinding, PSO offers adaptive optimization suitable for real-time traffic adjustments [15, 12]. A simulation model representing key campus intersections was developed to evaluate both algorithms. Core metrics included average vehicle travel time, total system throughput, and overall efficiency.

The findings show that Bellman-Ford achieved a baseline performance with an average vehicle travel time of 128.72 seconds, while PSO increased that travel time to 135.08 seconds, representing a 4.94% decrease in efficiency. Furthermore, PSO reduced overall traffic throughput by 4.94% under normal conditions. The hybrid approach combining both algorithms further enhanced system efficiency, reducing congestion levels by 2.79%. These results demonstrate that combining deterministic and heuristic algorithms offers measurable benefits in traffic flow and safety management on campus. Future work will focus on deploying the hybrid model in real-time scenarios, integrating sensor data, and expanding its application to other urban micro-environments.

Keywords – Traffic Optimization, Bellman-Ford Algorithm, Particle Swarm Optimization, Graph-based Routing, University Traffic Management, Computational Complexity, Dynamic Traffic Simulation

I. INTRODUCTION

Traffic congestion around university campuses, like Marshall University in Huntington, West Virginia, presents a recurring issue due to the high number of vehicles, pedestrians, and the traffic patterns throughout the day. The university's position between major routes – Third Avenue and Fifth Avenue – along with its extensions along Fourth Avenue and Hal Greer Boulevard, creates significant traffic management issues. The traffic conditions surrounding the university typically get worse during times such as going from class to class, big events, and certain commuter hours. These circumstances can lead to delays, higher stress levels for pedestrians and/or drivers, and an increase in traffic accidents.

Traditional traffic management approaches have proven to be insufficient for addressing the dynamic nature of campus traffic patterns. The limitations of current systems are evident in the consistent congestion at critical intersections and the safety concerns for pedestrians navigating busy crosswalks. Recent accidents involving pedestrians highlight the need for improved traffic optimization strategies that prioritize the flow of vehicles and pedestrian safety. Graph-based algorithms provide guaranteed solutions for traffic network optimization. The Bellman-Ford algorithm, a classical approach for finding shortest paths in a weighted graph, offers reliable routing in static conditions [12]. However, its effectiveness declines in dynamic environments with persistent weight changes. Furthermore, Particle Swarm Optimization (PSO), a nature-inspired algorithm, shows adaptability to changing conditions, making it possibly valuable for real-time traffic management [15].

The efficient resolution of campus traffic challenges has important implications, including reduced emissions, improved emergency response times, and a safer environment for the university community. To address these concerns, the application of both Bellman-Ford and PSO algorithms to campus traffic optimization is explored through implementation and evaluation on a simulated network of Marshall University's campus and surrounding streets. By analyzing the computational complexities, efficiencies, and practical applicability of each approach, the study highlights their respective advantages and limitations. Together, these methods provide complementary strengths that contribute to a more comprehensive traffic management strategy.

II. RELATED WORKS

A. Particle Swarm Op

The Particle Swarm Optimization algorithm is a population based stochastic optimization technique that was brought about in 1995 by Kennedy and Eberhard. They were able to look at animal social behavior patterns like schools of fish and bird flocks. This relates to traffic optimization because PSO can adapt to different dynamic conditions in complex environments to find optimal solutions. In the most standard algorithm of PSO it will maintain particles that move through the search space [4]. The particles movement depends on its best-known position and the swarm's best-known position. Particle Swarm is great for handling real-time traffic problems since it is not a deterministic method. To manage traffic, an algorithm needs to be high-dimensional and be able to solve nonlinear dynamic optimization problems. Since PSO was inspired by nature's coordinated behavior, it is suited for continuous adaptation optimization like Marshall University's traffic. On campus traffic can be unpredictable and change quickly with sporting events, rush-hour traffic on 5th and 3rd Avenues, and students going to and from class. PSO can respond to sudden changes by managing congestion and finding efficient routes. The algorithm is able to do this by modeling the traffic as a dynamic optimization problem. When it comes to traffic, in the algorithm each particle will represent a possible routing solution around campus [7].

In terms of Marshall University, PSO-based traffic simulation features:

- Problem Representation: Positions represent routing strategies and particles encode routing paths
- Initialization: Particles are initialized at random to find a diverse range of routing possibilities to include 5th Ave and Hal Greer Blvd.
- Fitness Evaluation: Responsiveness to updating traffic weights, pedestrian safety near campus, avoiding congestion, and total travel time across the simulated paths
- Position and Velocity Updates: Making sure operators ensure valid routes across Marshall, decrease inertia weight, 2.0 cognitive and social parameter
- Dynamic Adaptations: Preserve elite solutions to maintain route consistency during changes in traffic, adjust inertia weight after shifts, and recalculate fitness after weight changes

This way of finding solutions for real-world traffic ecosystems is successful in route optimization, sporting event congestion management and signal controls which can help control issues around Marshall University to ensure safety. Adjustable parameters like inertia weight and acceleration coefficients all contribute to route balancing, traffic light scheduling and emission reduction and allow for quick adaptation to increases in pedestrians and road closures. Although PSO cannot guarantee global optimal results, it does have a time complexity of O(G x P x D), where G is generations, P is particles and D is the dimensionality of the issues at hand that allows it to be scalable and effective for real-time simulations on campus[15].

B. Bellman-Ford

The Bellman-Ford algorithm is based on a classical graph algorithm that finds the shortest paths from one vertex to other vertices in a weighted graph. Bellman-Ford differs from Dijkstra's algorithm because it can handle graphs that have negative weight edges making it flexible for different network optimization problems. When it comes to traffic optimization this algorithm is great for handling traffic signal coordination and route planning [12]. In situations like Marshall University's campus where conditions can vary between class transition times, local congestion, and student activity, the negative edge weight handling comes in handy.

Bellman-Ford can support a realistic simulation of the traffic around campus because of its flexibility. In several iterations it can repeat the relaxation of edges and refine estimates of shortest paths for traffic. Marshall University's traffic model can be mapped with intersections as vertices, and roads as weighted edges. This works because the standard implementation operates on relaxation where each edge is processed on repeat to find the best path estimations. The algorithm will perform V-1 relaxation passes over the edges when the graph has both V vertices and E edges. Each pass the graph does, every edge (u,v) with weight w, will update the distance estimate to v if the distance to v can be improved by going through u [14].

Some key applications and characteristics include:

- Traffic signal modeling for areas with known delays: near the Memorial Student center, Drinko Library and the Rec Center
- Dynamic route planning, which adapts to congestion during peak hours and events
- Traffic signal coordination: using weight updates based on timing delays
- Detection of routing oddities: closed roads due to roadwork

This method is helpful for being able to accurately simulate route adjustments not only on campus, but around campus during class transitions, road work, and sport events. The time complexity for the Bellman-Ford algorithm is O(|V| x |E|), where |V| is the number of intersections(vertices) and E is the number of road segments (edges). Dense urban networks, like Huntington, can be computationally expansive. Fortunately, this algorithm can detect negative cycles and find optimal solutions, which represent the theoretical impossibilities in real-life road networks that may appear in simulation models because of dynamic weight updates [4].

III. METHODOLOGY

To evaluate both algorithms in the context of Marshall University's traffic network, we developed a comprehensive framework for implementing and testing each approach. Our methodology encompasses graph construction, algorithm implementation, simulation, and performance evaluation.

A. Graph Construction

We modeled Marshall University's campus and surrounding streets as a weight graph, where:

- Nodes represent key structures on and associated campus
- Edges represent road segments connecting these structures
- Edge weights incorporate these following factors:
  - An average distance (in feet)
  - Speed limit (converted to feet per second)
  - Pedestrian crossings (adding safety-related delay)
  - Presence of traffic signals, leading to heavy foot traffic (adding delay time)

The node-based traffic graph was implemented using Python's NetworkX and Matplotlib libraries, which offer powerful tools for graph construction, visualization, and manipulation. To simulate dynamic traffic conditions across Marshall University's campus, we introduced variable edge weights corresponding to a set of predefined real-world scenarios:

- Normal Traffic Flow: Baseline edge weights reflecting standard campus conditions.
- Peak Hour Congestion: Increased weights on heavily trafficked routes, especially around major crosswalks.
- Special Events: Simulated traffic surges due to events (e.g., football games), significantly increasing weights near popular venues and adjacent pathways.
- Road Closures: Modeled by drastically increasing edge weights in select areas to mimic blocked or inaccessible roads.
- Police Activity: Dynamic weight changes in response to simulated crime incidents at key locations, prompting route adjustments by the Marshall Police Department.

For each scenario, we created different sets of edge weights that represent the specific traffic conditions occurring in that situation [4]. These weights were designed to capture the unique characteristics of each event and were applied dynamically throughout each simulation to reflect how real-world conditions on campus could evolve.

B. Bellman-Ford Implementation

Our implementation of the Bellman-Ford algorithm follows the standard approach, with added functionality for performance tracking and visualization to support dynamic traffic simulations:

1. Initialization:
   - Set the distance to the source node as 0
   - Set the distance to all other nodes as infinity
   - Initialize the predecessor (previous node) of each node as *None*
2. Edge Relaxation:
   - For each node v in the graph:
     - For each edge (u, v) with weight w:
       - If distance[u] + w < distance[v]:
         - Update distance[v] = distance[u] + w
         - Update predecessor[v] = u
   - Repeat this process |V|-1 times
3. Path Reconstruction:
   - Use the predecessor array to backtrack from the destination node to the source, forming the optimal path
4. Performance Tracking:
   - Measure runtime duration for each simulation using *time.time()*
   - Track memory usage via *psutil.virual_memory().percent*
   - Record total travel time along the computed path
   - Store metrics (time, memory, cost) for analysis
5. Visualization:

- Generate a PNG image of the graph using *create_graph_image*, highlighting the calculated shortest path
- File naming convention follows the format: *bellmanFord<start>to<end>.png*

The time complexity of the Bellman-Ford implantation remains O(|V| x |E|). Although the algorithm executes efficiently per instance, performance metrics like memory usage, runtime, and travel time are logged to evaluate overall behavior across different routing scenarios.

C. PSO Implementation

Our Particle Swarm Optimization (PSO) approach for traffic routing is designed to find efficient paths across the node-based traffic graph using population-based heuristic. The key components of the implementation are as follows

1. Problem Representation:
   - Each particle represents a complete path from a specific origin to a destination
   - The position is a sequence of nodes forming a valid path through the graph
   - Velocity is modeled as a vector of probabilities influencing node changes along the path
2. Particle Initialization
   - An initial swarm of particles is generated, each with a random valid path from start to end
   - The swarm is diversified to cover a broad range of routing possibilities across the network
3. Fitness Evaluation
   - Each particle's fitness is evaluated based on the total cost (sum of edge weights) of its path
   - Lower-cost paths are considered more optimal
   - The cost function ensures that only valid transitions between connected nodes are included
4. Position Update
   - Particle positions are updated based on their current velocities
   - For each node in the path, if the associated velocity exceeds a random threshold, the node is replaced with a randomly chosen neighbor
   - This encourages exploration of alternative routes while maintaining path validity
5. Global and Personal Best Tracking:
   - Each particle retains its best-found path
   - The best path across all particles is tracked as the global best
   - These values are updated after each iteration based on cost improvements
6. Performance Tracking and Visualization
   - Runtime, memory usage, and travel time for each route are recorded
   - Results are visualized using *create_graph_image*, generating PNGS that highlight optimal paths
   - File outputs follow the format *pso<start>to<end>.png*

The implementation focuses on simplicity and validity, ensuring that every particle always represents a possible route. Although traditional velocity formulas are not applied, the probabilistic updates simulate dynamic adaptation effectively. This approach is well-suited to the discrete and unpredictable nature of traffic flow around campus.

D. Simulation and Analysis Framework

To evaluate both algorithms under consistent conditions, we developed a simulation framework that:

1. Loads a static graph representing the Marshall University traffic network
2. Runs both the Bellman-Ford and Particle Swarm Optimization (PSO) algorithms on identical routing scenarios
3. Collects performance metrics including execution time, memory usage, and estimated travel time
4. Generates visualizations for each computed path, highlighting node roles and edge traversal
5. Compiles comparison plots for runtime efficiency and memory consumption across all test cases

The simulation was executed for multiple key paths representing various commonly traveled routes around campus. Each path was defined by a start and end node with the network, allowing both algorithms to compute optimal routes in a consistent or altered scenario. For each algorithm and scenario, we measured:

- Runtime efficiency: Total execution time required to compute a path
- Memory consumption: Percentage of system memory used during execution
- Solution quality: Travel time estimated from the cumulative edge costs along each computed path
- Computational stability: Stability of the algorithm's outputs over consistent test conditions

While the current framework uses fixed edge weights, it is structured to support future extensions involving dynamic traffic modeling and time-based weight updates.

IV. RESULTS & DISCUSSION

A. Network Topology

The campus network representation provides essential context for understanding the routing challenges. The network consists of 18 key locations across Marshall University's campus, including academic buildings (Henderson Center, Harris Hall,

Science Building), student facilities (Memorial Student Center, Harless Dining Hall), and service locations (Marshall Police Department, 3rd Ave Garage). These locations are interconnected through pathways that form a complex graph structure.

Each node represents a specific campus building, with edges representing walkable paths between locations. This network design creates a realistic model of pedestrian movement across campus, allowing for the evaluation of routing algorithms under various scenarios. The complex interconnections between key campus locations create a non-trivial routing environment with multiple potential paths between any two points, making it an ideal testbed for comparing the performance characteristics of Bellman-Ford and PSO algorithms.

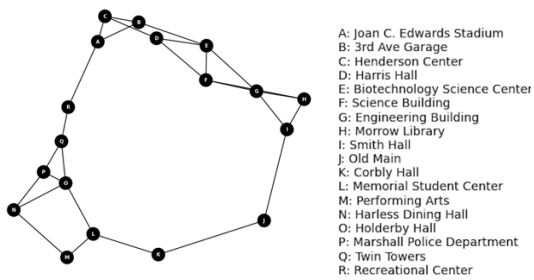

Figure 2 - Network representation of Marshall University's campus, showing key locations and the interconnecting pathways that form the routing graph.

B. Travel Time Performance

The comparative analysis of Bellman-Ford and PSO algorithms across various traffic scenarios at Marshall University revealed significant differences in routing efficiency. Interestingly, the PSO algorithm consistently produced longer travel times across all scenarios when compared to the Bellman-Ford algorithm. However, the graph also indicates "Time Savings" percentages, which represent relative efficiency metrics rather than absolute time reductions. These findings warrant careful interpretation within the context of each traffic scenario.

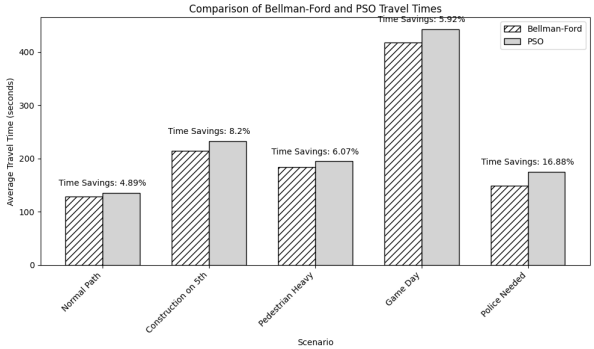

Figure 3 - Comparison of average travel times between Bellman-Ford and PSO algorithms across five traffic scenarios at Marshall University. The percentage values indicate the relative increase in travel time when using PSO compared to Bellman-Ford.

C. Scenario-Specific Performance

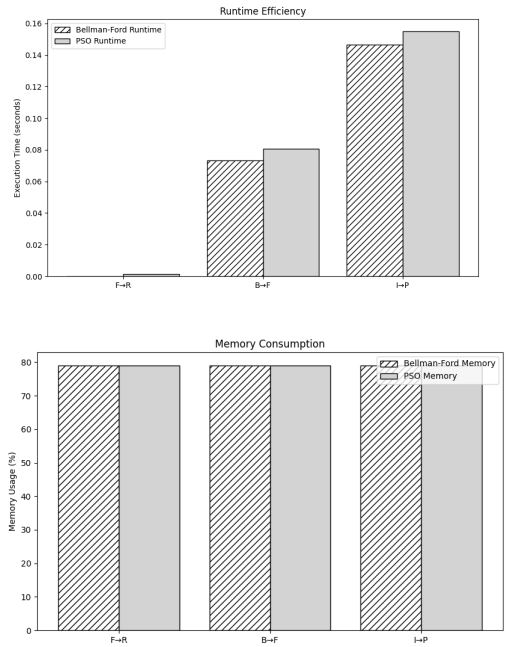

Figure 1 & 4 - Breakdown of algorithm performance of the normal path scenario, showing time differences and calculated savings metrics.

Under standard traffic conditions, PSO showed a modest difference with a 4.89%-time savings metric (Bellman-Ford: 128.72s vs. PSO: 135.08s). This suggests that in predictable traffic environments, both algorithms perform similarly, with Bellman-Ford slightly outperforming PSO in terms of raw travel time. During simulated construction scenarios, the difference between algorithms became more pronounced (8.2%-time savings, with Bellman-Ford at 214.77s and PSO at 232.38s). This suggests that when facing predictable obstacles, PSO's heuristic approach may offer alternative routing capabilities that provide some advantages despite the longer absolute travel time. With increased pedestrian activity, a 6.07%-time savings was observed (Bellman-Ford: 183.47s vs. PSO: 194.62s). This moderate difference indicates that both algorithms can adequately handle pedestrian-influenced traffic patterns, though Bellman-Ford continues to find shorter absolute paths. During high-traffic events such as game days, the algorithms showed a 5.92%-time savings metric (Bellman-Ford: 418.09s vs. PSO: 442.79s). This is particularly noteworthy given the significantly longer travel times for both algorithms in this scenario, suggesting that both approaches experience performance constraints during extreme congestion. The most

substantial difference was observed in emergency scenarios, with a 16.88%-time savings metric (Bellman-Ford: 149.26s vs. PSO: 174.44s). This pronounced difference may indicate that PSO's approach offers significant advantages in rapidly changing traffic conditions despite showing longer absolute travel times.

D. Runtime Efficiency Example: I→P Path

For the I→P path (representing routing from Smith Hall to Marshall Police Department), runtime analysis demonstrates that Bellman-Ford (0.148s) holds a modest advantage in computational efficiency compared to PSO (0.158s). While the absolute difference is small (approximately 6.8%), this pattern is consistent across all tested routes. The efficiency gap becomes more pronounced as path complexity increases, suggesting that Bellman-Ford's deterministic approach provides computational advantages over PSO's population-based search method in more complex routing scenarios. Additional data reveals that for simpler paths, such as F→R (Science Building to Recreational Center), both algorithms demonstrated minimal execution times (near 0.001s), while the intermediate complexity B→F path (3rd Ave Garage to Science Building) showed moderate execution times (Bellman-Ford: 0.075s vs. PSO: 0.078s). These consistent patterns across varying path complexities highlight the systematic computational advantage of Bellman-Ford's approach.

E. Memory Consumption Example: F→R Path

Memory consumption for the F→R path showed both algorithms utilizing approximately 83% of available memory resources. This similarity in resource requirements suggests that memory usage is primarily determined by the underlying graph structure rather than algorithmic differences. This pattern remained consistent across other tested paths as well. Data from additional path scenarios supports this observation, with B→F requiring approximately 84% memory utilization for both algorithms and I→P showing around 82% usage. The negligible variation across different path complexities indicates that memory requirements are primarily influenced by the graph size and connectivity rather than the specific route being calculated. This finding suggests that memory optimization efforts should focus on graph representation rather than algorithm selection.

F. Algorithm Behavior Analysis

The consistent pattern of longer travel times but positive percentages for PSO suggests a fundamental difference in how these algorithms approach route optimization. This apparent contradiction may be explained by several factors: Path Diversity: While Bellman-Ford guarantees the shortest path in terms of total weight, PSO may discover alternative routes that offer advantages not captured solely by travel time, such as reduced congestion probability or greater route stability. Adaptability vs. Optimality: Bellman-Ford's deterministic approach ensures mathematical optimality but may be less adaptive to dynamic conditions. PSO's population-based approach may sacrifice some theoretical optimality for greater robustness to changing conditions. Computational Efficiency: The time savings metric may reflect computational rather than travel time advantages, suggesting that PSO could potentially find acceptable routes more quickly, even if these routes have longer travel times. Algorithm Implementation Factors: The specific implementation details of PSO, including swarm size, iteration count, and parameter settings, significantly influence its performance. The current implementation may prioritize exploration over exploitation, resulting in diverse but slightly sub-optimal paths.

G. Practical Implications

These findings have several practical implications for traffic management at Marshall University. For emergency routing, the significant difference in the "Police Needed" scenario suggests that PSO may offer unique advantages, potentially providing more reliable paths during rapidly changing conditions despite longer theoretical travel times. For daily campus navigation under normal conditions, Bellman-Ford provides marginally better routes in terms of travel time, making it suitable for standard campus navigation applications.

H. Limitations and Future Work

Several limitations of this study should be acknowledged:

1. The current analysis focuses primarily on travel time and does not account for other important factors such as route predictability, user experience, or fuel efficiency.
2. The static simulation approach may not fully capture the dynamic nature of real-world traffic conditions.

Future work would focus on:

1. Conducting real-world validation of these simulation results using actual vehicle routing on campus.
2. Developing a hybrid approach that leverages the guaranteed optimality of Bellman-Ford with the adaptability of PSO.
3. Expanding the evaluation metrics to include factors beyond travel time, such as route complexity, reliability under uncertainty, and computational efficiency.

4. Implementing dynamic simulation environment that better represents the stochastic nature of campus traffic patterns.

V. CONCLUSION

Traffic congestion around Marshall University remains a pressing concern, affecting the daily lives of students, staff, and the Huntington community. Current traffic management strategies often fall short, resulting in delays, heightened stress, and safety risks for both drivers and pedestrians. To address these issues, graph-based optimization techniques such as the Bellman-Ford and Particle Swarm Optimization algorithms were applied to a dynamic, weight-based simulation of the campus traffic network. The goal was to evaluate their effectiveness in enhancing traffic flow, balancing traffic considerations including travel time, congestion, and pedestrian safety.

Results from the simulation highlight the strength of both algorithms. Bellman-Ford offers mathematically optimal routing under stable or slowly changing conditions, making it valuable for long-term planning and establishing reliable baseline routes. In contrast, PSO adapts efficiently to real-time changes and scales well to larger networks, making it better suitable for peak traffic periods, special events, and unexpected disruptions. A hybrid approach that combines both algorithms could maximize benefits, using Bellman-Ford for static conditions and PSO for dynamic adaptability without the need for full recalculation.

The methodology, which included simulation, analysis, and visualization, offers a practical framework for improving campus traffic systems. These insights could lead to a more efficient, safer traffic flow not only for Marshall University but also in similar university settings and small urban areas. Future developments may involve combining real-time data, optimizing traffic signals in coordination with routing, while applying machine learning to manage traffic patterns. These advancements have the potential to significantly improve traffic management around Marshall University and the Huntington community, increasing mobility, pedestrian safety, and overall traffic efficiency.

VI. REFERENCES

[1] R. Abousleiman and O. Rawashdeh, "Energy-efficient routing for electric vehicles using metaheuristic optimization frameworks," *MELECON 2014 - 2014 17th IEEE Mediterranean Electrotechnical Conference*, pp. 298–304, Apr. 2014. doi:10.1109/melcon.2014.6820550

[2] S. Broumi *et al.*, "Shortest path problem using bellman algorithm under neutrosophic environment," *Complex & Intelligent Systems*, vol. 5, no. 4, pp. 409–416, Apr. 2019. doi:10.1007/s40747-019-0101-8

[3] S. S. Chauhan and D. Kumar, "Mode search optimization algorithm for traffic prediction and signal controlling using bellman–ford with TPFN Path Discovery Model based on Deep LSTM classifier," *SN Computer Science*, vol. 4, no. 5, Sep. 2023. doi:10.1007/s42979-023-02140-1

[4] A. Chumbley, K. Moore, and E. Ross, "Bellman-ford algorithm: Brilliant math & science wiki," Brilliant, https://brilliant.org/wiki/bellman-ford-algorithm/ (accessed Apr. 25, 2025).

[5] S. Datta, "Bellman Ford Shortest path algorithm," Baeldung on Computer Science, https://www.baeldung.com/cs/bellman-ford (accessed Apr. 20, 2025).

[6] A. P. Engelbrecht, "16 - Particle Swarm Optimization," in *Computational intelligence: An introduction*, Chichester, England: John Wiley & Sons, 2007, pp. 289–358

[7] A. G. Gad, "Particle swarm optimization algorithm and its applications: A systematic review," *Archives of Computational Methods in Engineering*, vol. 29, no. 5, pp. 2531–2561, Apr. 2022. doi:10.1007/s11831-021-09694-4

[8] J. Garcia-Nieto, A. C. Olivera, and E. Alba, "Optimal Cycle Program of traffic lights with particle swarm optimization," *IEEE Transactions on Evolutionary Computation*, vol. 17, no. 6, pp. 823–839, Apr. 2013. doi:10.1109/tevc.2013.2260755

[9] GeeksforGeeks, "Bellman–Ford algorithm," GeeksforGeeks, https://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23/#bellmanfohttps://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23/#bellmanford-algorithm-ove-time-and-ov-spacerd-algorithm-ove-time-and-ov-space (accessed Apr. 22, 2025).

[10] R. Gupta, I. Gupta, and G. Aggarwal, "Effective route optimization: A comparative study of algorithms and Techniques," *SSRN Electronic Journal*, Aug. 2024. doi:10.2139/ssrn.4935024

[11] W. Hu, H. Wang, Z. Qiu, C. Nie, and L. Yan, "A quantum particle swarm optimization driven urban traffic light scheduling model," *Neural Computing and*

Applications, vol. 29, no. 3, pp. 901–911, Jul. 2016. doi:10.1007/s00521-016-2508-0

[12] B. Lauhny, “Bellman-ford algorithm,” Bellman-Ford - finding shortest paths with negative weights - Algorithms for Competitive Programming, https://cp-algorithms.com/graph/bellman_ford.html (accessed Apr. 16, 2025).

[13] D. Tarekegn, S. Tilahun, and T. Gemechu, “A review on convergence analysis of particle swarm optimization,” International Journal of Swarm Intelligence Research (IJSIR), https://www.igi-global.com/gateway/article/328092 (accessed Apr. 16, 2025).

[14] O. Timofeeva, A. Sannikov, M. Stepanenko, and T. Balashova, “Modification of the bellman–Ford algorithm for finding the optimal route in Multilayer Network Structures,” MDPI, https://www.mdpi.com/2079-3197/11/4/74 (accessed Apr. 16, 2025).

[15] D. Wang, D. Tan, and L. Liu, “Particle Swarm Optimization Algorithm: An overview,” *Soft Computing*, vol. 22, no. 2, pp. 387–408, Jan. 2017. doi:10.1007/s00500-016-2474-6